

LECTURE 2

Working Effectively with Generative AI

Context, task design, multimodality and reliable interaction

Armando Vieira

Lecture 2

A better prompt cannot rescue an undefined task

Generative AI becomes reliable when we design the task, context, output and verification together.

The question for this lecture

How do we turn a vague request into a structured interaction that can be checked, improved and reused?

What this lecture changes

- Replace “write a good prompt” with **design a precise task specification**.
- Treat context as a limited working resource, not as an unlimited company memory.
- Make evidence, uncertainty and abstention part of the requested output.
- Use multimodal inputs when they contain information that text alone loses.
- Compare models and tools using explicit trade-offs rather than personal preference.
- Recognize when rules, databases, statistical models or human judgment are better choices.

The four-hour session has two distinct halves

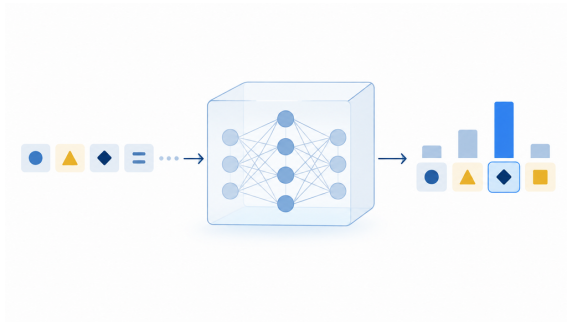
Concept block – 120 minutes

- What an LLM is and is not
- Tokens and context limits
- Task and context design
- Structured outputs and abstention
- Multimodal inputs
- Choosing the right tool

Practical block – 120 minutes

- Repair a weak business request
- Add role-specific context
- Build an output schema
- Compare two tools or models
- Red-team the result
- Submit a reusable specification

A language model is a probability engine for language



At generation time, the model

- receives prior tokens;
- scores plausible next tokens;
- selects or samples one;
- repeats until it stops.

Business consequence. Fluent language does not guarantee facts, intention or self-detected mistakes.

Task design and verification make this useful capability dependable in a workflow.

Fluency and reliability are different properties

A fluent answer may be	What the organization still needs	Design response
Well written	Factually supported	Ask for sources and inspect them
Specific	Based on the wrong context	Supply authoritative context
Confident	Uncertain or fabricated	Require uncertainty and abstention
Consistent	Consistently wrong	Evaluate against representative cases
Helpful in isolation	Harmful in the workflow	Test downstream decisions and actions

The model does not see your business

Usually missing from a generic request

- Organization-specific definitions
- Current policies and exceptions
- The real decision owner
- Customer and employee constraints
- Historical failures and edge cases
- What must happen when information is missing

Result

The model fills gaps with plausible general knowledge. That is often exactly what a business process cannot afford.

Context is part of the system.

A context window is working memory

Concept	Meaning	Business implication
Context window	Information available for the current interaction	Relevant facts must fit and be selected
Memory	Information retained across interactions or tasks	Retention requires a deliberate storage policy
Retrieval	Selecting information from an external source	Search quality becomes part of answer quality
Context engineering	Designing what the model receives and how it receives it	Context needs ownership, versioning and evaluation

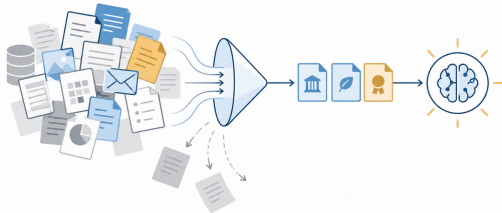
A larger context window increases capacity. It does not automatically create understanding, relevance or organizational memory.

More context can still produce less useful reasoning

Common failure modes

- The relevant fact is buried among similar documents.
- Contradictory versions are presented without resolution.
- Long documents contain irrelevant detail.
- The model gives equal attention to high- and low-value material.
- The user cannot inspect which context shaped the answer.

Better design: Select, label and prioritize context. Keep authoritative policy separate from examples and historical material.



Non-determinism is a design constraint

Two runs may differ because of

- sampling and decoding settings;
- model or system updates;
- small changes in context;
- ambiguous instructions;
- external tools and retrieved documents.

Reliable response

Test important tasks repeatedly, define acceptable variation, log the input context, and compare outputs against a stable rubric.

A single impressive answer is a demonstration. A distribution of tested answers is evidence.

Prompting is task design

Good instructions specify the work around the model.

Goal

What decision, action or artifact should improve?

Context

What facts, definitions, documents and history are relevant?

Constraints

What must the system obey, avoid or escalate?

Examples

What does a good result and a bad result look like?

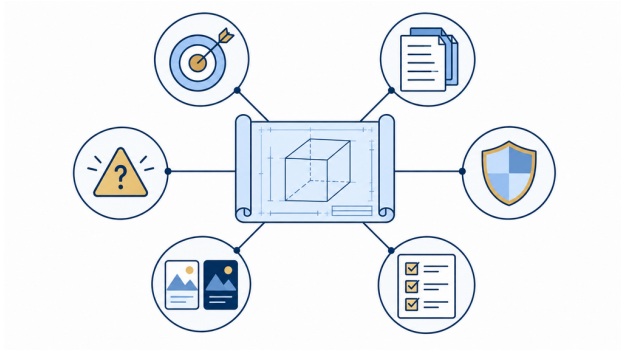
Output

What fields, format, length and audience are required?

Uncertainty

When should the system qualify, ask, defer or abstain?

A task specification makes choices explicit



A reusable specification names

- the intended outcome;
- authoritative context;
- constraints and escalation rules;
- examples defining acceptable work;
- required output fields;
- the uncertainty path.

What remains unnamed becomes an implicit model decision.

A vague instruction leaves the important decisions implicit

Weak request

Write a response to this customer complaint. Be helpful and professional.

What remains unspecified?

- Which policy applies?
- What compensation is allowed?
- Which facts are confirmed?
- Who is the customer and what tone is appropriate?
- When should the case be escalated?
- How should uncertainty be represented?

A context specification makes the task testable

Improved request

You are assisting a NovaRetail customer-service agent. Use only the approved return policy and the confirmed order information below. Draft a reply of at most 120 words. State the relevant policy rule, separate confirmed facts from assumptions, and do not promise a refund unless the policy clearly permits it. If information is missing or contradictory, mark the case ESCALATE and explain why. Return the result using the specified fields.

Still required: the policy text, the order record, the output schema, examples, and the evaluation rubric.

Examples define the boundary of acceptable work

Example type	What it teaches	Risk if omitted
Normal case	The expected workflow	The model may overcomplicate routine cases
Ambiguous case	When to ask or escalate	The model may invent missing information
Negative case	What the system must reject	The model may accept unsafe requests
Policy exception	Where a general rule changes	The model may apply a rule mechanically
Good and bad output	What quality means in practice	“Helpful” remains subjective

Design question

Which examples would expose the model's likely failure modes before deployment?

Constraints protect the workflow

Source constraints

Use specified documents, fields and versions. Do not silently replace missing evidence with general knowledge.

Action constraints

Draft, recommend or act only within explicit permissions. Escalate high-impact or irreversible cases.

Output constraints

Return required fields, valid values, evidence and uncertainty. Make missing information visible.

Constraints reduce the space of plausible answers. They make the system easier to evaluate and harder to misunderstand.

Structured outputs turn language into a process

Free-form response

The customer probably qualifies for a refund, although more information may be needed.

Hard to route, compare, audit or evaluate automatically.

Structured response

```
decision: ESCALATE
policy_rule: R-14
confirmed_facts: [ ... ]
missing_information: [ ... ]
evidence: [ ... ]
confidence: LOW
draft_reply: "..."
```

Every answer needs an uncertainty path

A trustworthy system shows what it knows and what it cannot establish.

State	Meaning	System response
Confirmed	Directly supported by an authoritative source	Proceed within policy
Inferred	Plausible interpretation from available evidence	Label as inference and request review
Missing	Necessary information is unavailable	Ask for the missing information
Contradictory	Sources or records disagree	Escalate or resolve the conflict
Unsafe	The action could cause unacceptable harm	Refuse, stop or route to a responsible person

Decompose complex work into checkable steps

1. **Extract:** identify relevant facts, entities and missing fields.
2. **Classify:** assign a bounded category using a documented taxonomy.
3. **Retrieve:** find the policy, record or evidence needed for the decision.
4. **Recommend:** propose an action and explain the basis for it.
5. **Draft:** produce the communication or artifact for the intended audience.
6. **Verify:** check facts, policy, format, risk and downstream consequences.

Why decompose?

A single prompt can hide several different error types. Separate steps make failures visible and controls easier to place.

Use the LLM where ambiguity matters

Tool	Good fit	Weak fit
Rules	Explicit, stable and auditable decisions	Open-ended interpretation
SQL or database	Exact retrieval, aggregation and joins	Unstructured meaning
Statistical model	Repeated prediction with measurable outcomes	One-off explanation of a novel document
LLM	Language, interpretation, transformation and flexible drafting	Exact arithmetic or guaranteed facts
Human expert	High-stakes judgment, accountability and exceptions	Large-volume repetitive preparation

Design principle

The strongest design often combines several tools.

Multimodal inputs change what the system can inspect

Input	Useful information	Typical business task
Text	Meaning, tone, instructions, claims	Summarize, classify, draft
Table	Values, relationships, trends	Analyze, reconcile, explain
Document	Layout, sections, signatures, visual hierarchy	Extract evidence, inspect compliance
Image	Objects, damage, labels, spatial arrangement	Triage, quality inspection, verification
Audio	Speech, pauses, speakers, acoustic context	Transcribe, summarize, detect issues
Screen or workflow	Visible state, controls and sequence	Assist a user, diagnose a process

A document is more than a long text string

Information that may be lost

- Table structure and row relationships
- Headers, footnotes and page references
- Signatures, stamps and visual emphasis
- Charts, diagrams and handwritten notes
- Which version is authoritative



Model choice is a business trade-off

Dimension	Question	Why it matters
Quality	Does it solve the task on representative cases?	A benchmark score may not reflect the workflow
Reliability	Does it behave acceptably across repeated runs?	Rare failures may dominate business cost
Latency	How quickly must the result arrive?	Waiting changes adoption and process design
Cost	What is the cost per successful task?	Retries, review and errors count too
Privacy	What data leaves the organization?	Data boundaries may determine the architecture
Control	Can we constrain tools, sources and actions?	Capability without control creates exposure

Choose the model after defining the acceptance test

Weak model comparison

- “Model A sounds better.”
- “Model B is cheaper.”
- “Model C is more intelligent.”

Useful comparison

- accuracy on difficult cases;
- evidence and citation quality;
- consistency and abstention;
- response time and cost;
- privacy and integration constraints.

Decision rule

Select the least expensive and least complex system that meets the acceptance criteria with adequate control.

The same task changes under different role perspectives

Role	Question about an AI customer-service assistant	Evidence expected
Business owner	Does it improve service at an acceptable cost?	Baseline, outcome and operating cost
Customer	Will the answer be accurate, fair and easy to challenge?	Policy, explanation and recourse
Employee	Does it reduce preparation work without removing judgment?	Workflow, workload and escalation path
Data analyst	Can the system be tested on representative cases?	Data quality, labels and error analysis
Regulator or security lead	Can access and accountability be demonstrated?	Permissions, logs, privacy and incident response

Practical lab: the context engineering challenge

**Turn one vague business request
into a reusable, testable interaction
specification.**

Team output

A context specification, an output schema with evidence and uncertainty, a model or tool comparison, and a short red-team report.

Case: NovaRetail customer service

Current situation

- 12,000 support requests each month
- 40% concern returns, refunds, delivery or warranty
- Average response time: 18 hours
- First-contact resolution: 62%
- 28% of cases are escalated

Available context

- Customer conversations and order records
- Return and warranty policies
- Product and delivery information
- Refund and escalation history
- Some outdated and contradictory documents

The COO proposes an AI system that answers customers and approves simple refunds. The team must decide what role AI should actually play.

Your starting request is intentionally weak

The COO's request

Build an AI customer-service assistant for NovaRetail. It should answer customers quickly, solve their problems and approve simple refunds automatically. Make it accurate, helpful and professional.

Team question

What must be specified before a model can do useful work safely?

Round 1: assign roles and expose hidden requirements

Each team assigns

- Business owner
- Customer representative
- Customer-service employee
- Data and evaluation lead
- Risk and governance lead

Each role must state

- What a successful result means
- What information is required
- What failure would be unacceptable
- What the system may and may not do
- What evidence would change the recommendation

Round 1: build the context specification

Goal	Which customer-service outcome or decision should improve?
User	Who reads, checks or acts on the output?
Context	Which records, policies and definitions must be supplied?
Constraints	Which actions require approval or escalation?
Examples	Which ordinary, ambiguous and unsafe cases should be included?
Output	Which fields must the result contain?
Uncertainty	What should happen when information is missing or contradictory?

Deliverable

A one-page specification that another team can use without guessing your intentions.

Round 2: design an output schema

The schema should support routing, review and evaluation.

```
case_id: string
request_type: [return, refund, delivery, warranty, other]
confirmed_facts: list
missing_information: list
applicable_policy: string or null
recommended_action: [answer, request_info, escalate, refuse]
refund_authority: [none, agent, manager, automatic]
evidence: list of source_id and quoted_support
confidence: [high, medium, low]
draft_reply: string
```

The schema is part of the control system. It defines what the model must make visible.

Round 2: design the evidence fields

For every recommendation, record

- Which source supports it
- Which facts are directly confirmed
- Which facts are inferred
- Which policy rule applies
- What information is missing

A useful refusal

“I cannot recommend a refund because the order record does not confirm delivery status. Escalate to an agent after checking shipment record S-481.”

A refusal becomes useful when it explains the missing evidence and the next action.

Round 3: compare two model or tool choices

Criterion	Option A	Option B
Normal cases		
Ambiguous cases		
Evidence quality		
Abstention		
Latency and cost		
Privacy		
Integration		

Important

Include human review time, retries, corrections and the cost of an incorrect refund. Model price alone is not system cost.

Round 3: test difficult cases first

- The policy document is older than the order record.
- The customer asks for an exception and cites a promise that cannot be verified.
- The order contains two products with different return rules.
- The customer message contains an instruction aimed at manipulating the assistant.
- The delivery status is missing from one system and contradictory in another.
- The refund amount exceeds the automatic-approval limit.

Test question

Does the system become more cautious when the evidence becomes weaker?

Round 4: red-team the interaction

Attack the specification

- Can the model use an outdated policy?
- Can a customer force an exception?
- Can it invent a source or order fact?
- Can it expose another customer's data?

Attack the workflow

- Who notices a wrong recommendation?
- What happens after an automatic refund?
- Can an employee override the model?
- Is the decision reversible and traceable?

Red-team output

Three discovered failure modes, their likely consequence, and one design change for each.

The practical output is a reusable specification

- **Context specification:** goal, user, sources, constraints, examples and escalation rules.
- **Output schema:** fields for facts, recommendations, evidence, confidence and draft communication.
- **Test set:** normal, ambiguous, incomplete, contradictory and adversarial cases.
- **Comparison:** two models or tools evaluated using the same rubric.
- **Red-team report:** failures discovered and changes made in response.
- **AI-use record:** what the system suggested, what the team verified, changed, rejected or left uncertain.

Homework: improve one real task

Individual or team assignment

Choose a business task from your project and redesign the interaction using the lecture framework.

1. Show the original vague request.
2. Produce a context specification and structured output schema.
3. Add at least five test cases, including one where the system should abstain.
4. Compare two model or tool choices.
5. Document one failure and revise the design.
6. State what remains uncertain and what evidence you need next.

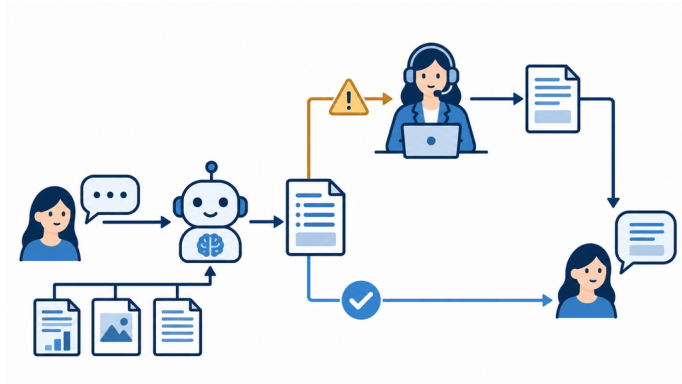
The standard for a good interaction

A good interaction makes the task clearer, the evidence visible and the failure safer.

Before the next lecture

Bring one task specification that another student can run, inspect and criticize without needing to guess your intentions.

A controlled workflow makes uncertainty actionable



The system needs two paths

- Proceed when evidence and authority are sufficient.
- Escalate when information is missing, contradictory or high-impact.

Human responsibility remains visible where the risk changes.

The model is only one part of the design

Task

A decision or action with a clear owner and outcome.

Context

Relevant, authoritative information selected for the task.

Control

Structured output, evidence, uncertainty and human responsibility.

Reliable AI begins before the model generates its first token.

Further reading

- Amershi et al. (2019), “Guidelines for Human–AI Interaction.”
- Lewis et al. (2020), “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.”
- Liu et al. (2024), “Lost in the Middle: How Language Models Use Long Contexts.”
- NIST (2024), *Artificial Intelligence Risk Management Framework: Generative AI Profile*.
- Dell’Acqua et al. (2023), “Navigating the Jagged Technological Frontier.”

The important lesson is methodological: define the task, supply the right context, make uncertainty visible, and test the complete workflow.